Library Management System – Technical Documentation

---

1. Introduction

1.1 Purpose

This document provides comprehensive technical documentation for the Library Management System, a production-grade REST API built with Java 22 and Spring Boot 3. The system enables library operations including book management, borrowing, returning, and advanced search capabilities.

1.2 Scope

The system implements:

- Complete book CRUD operations with soft delete
- Borrowing and return management with business rule enforcement
- Advanced search with pagination, sorting, and filtering
- Professional-grade testing, containerization, and CI/CD

1.3 Target Audience

This documentation is intended for:

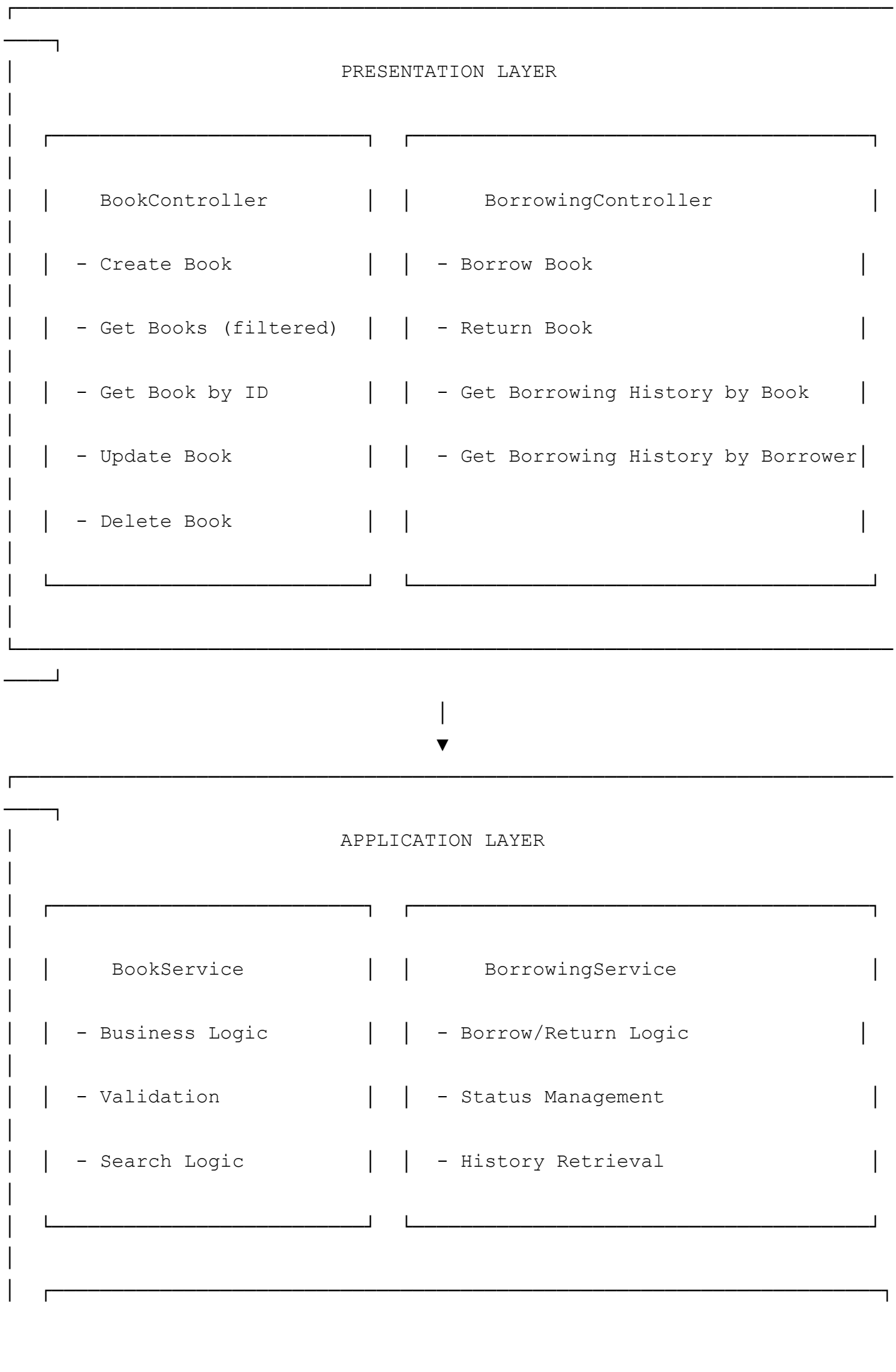
- Backend developers maintaining the system
- Technical leads reviewing the architecture
- DevOps engineers managing deployment
- Future contributors to the project

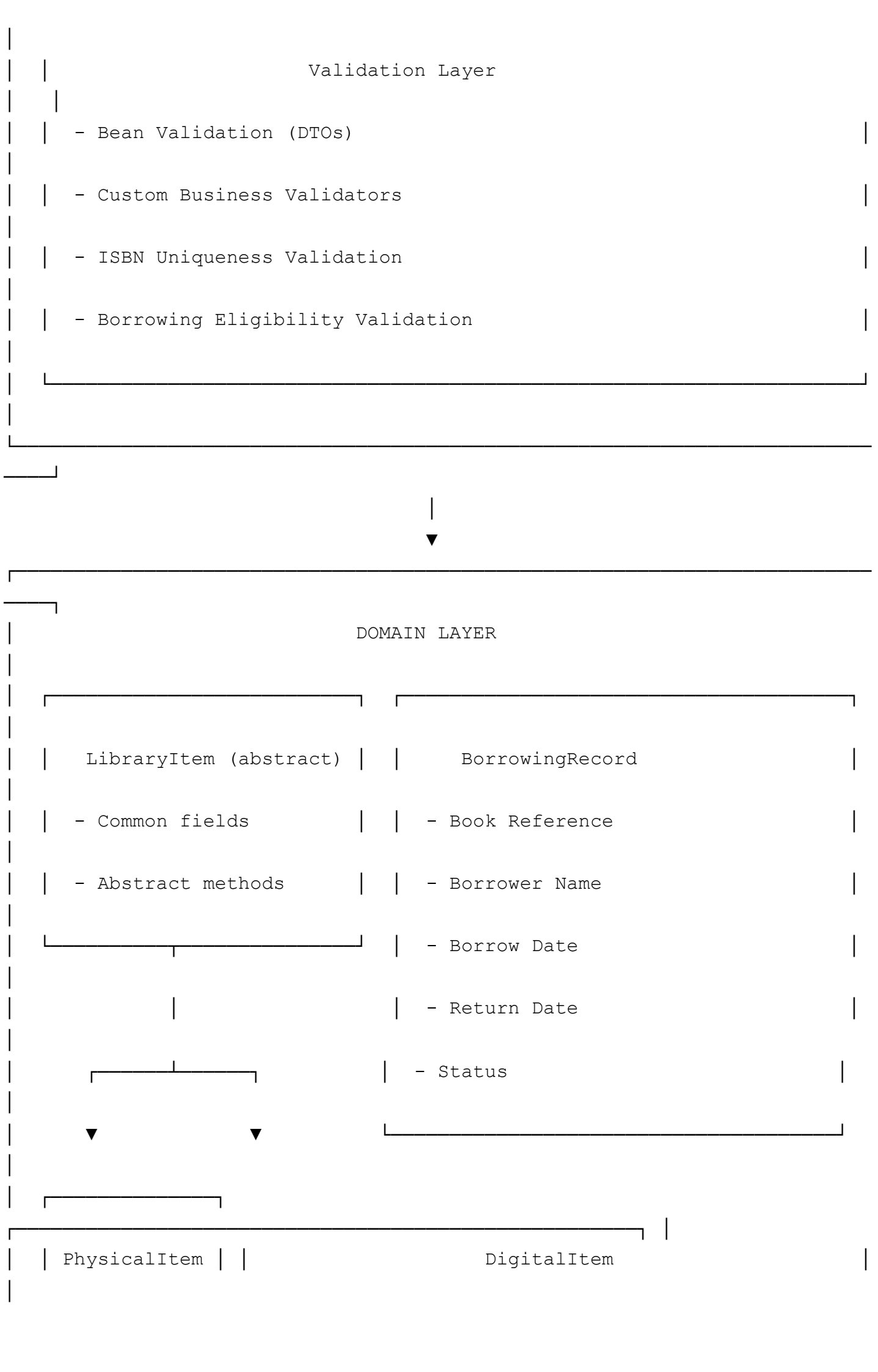
---

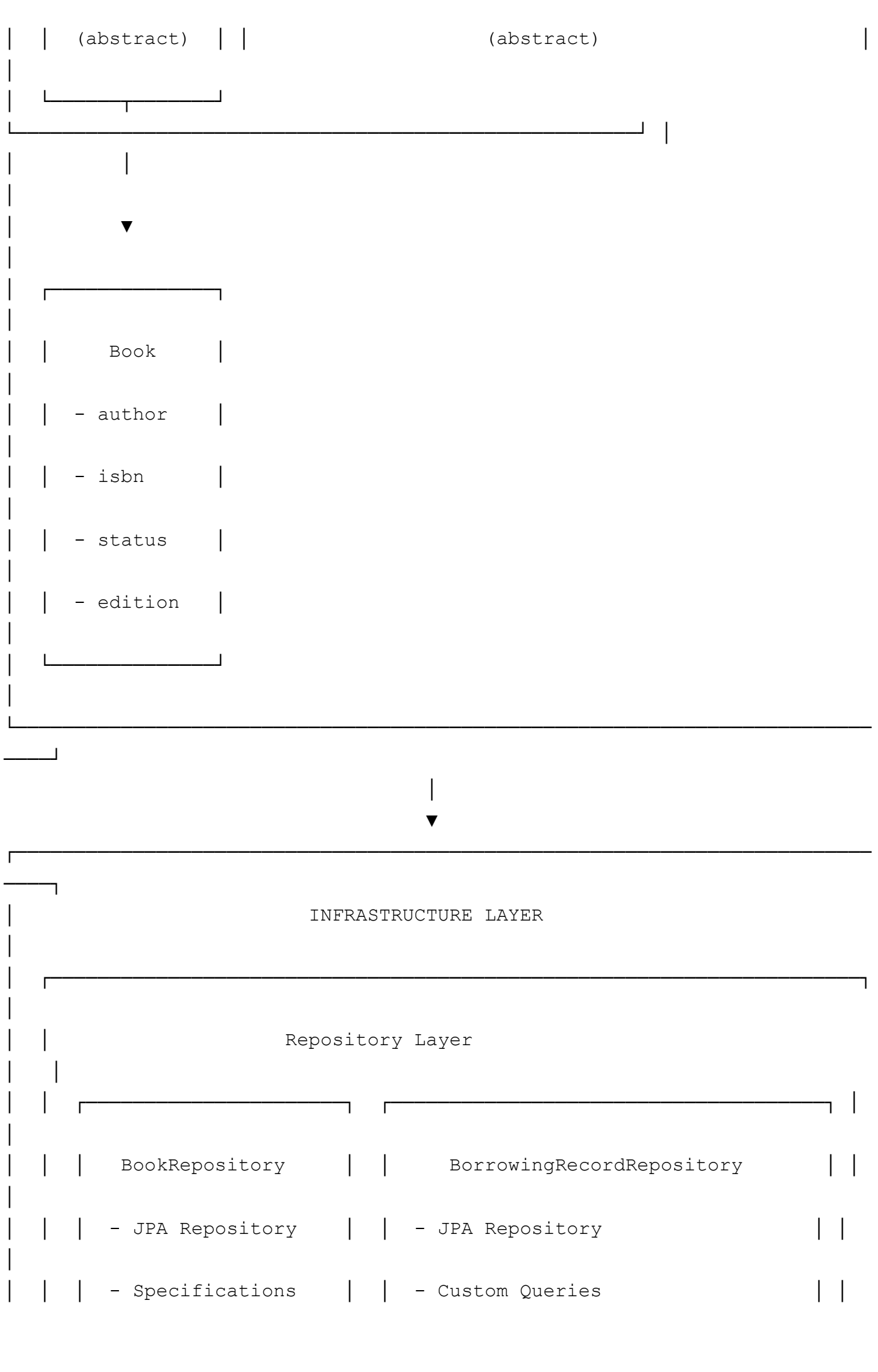
2. System Architecture

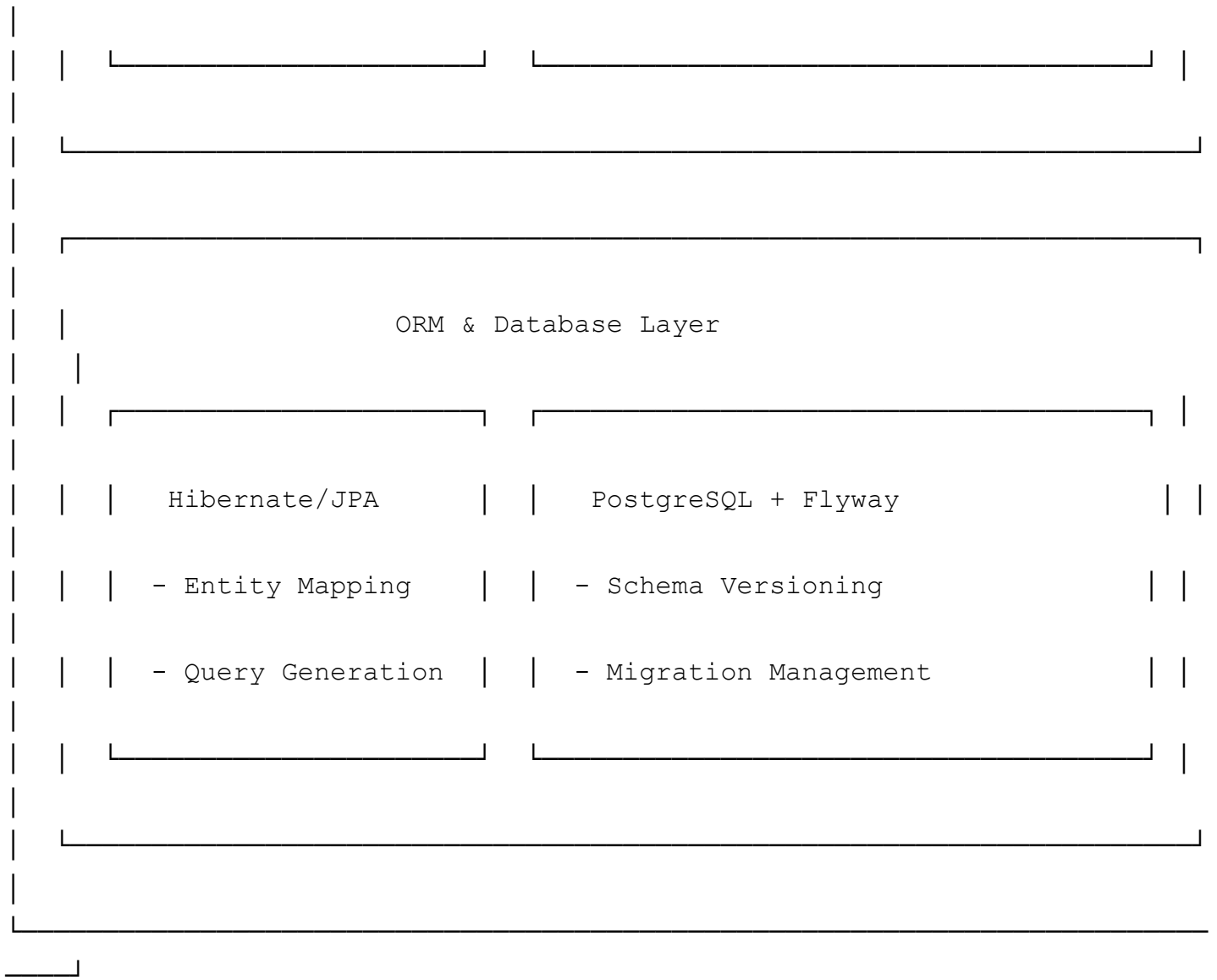
2.1 High-Level Architecture

The system follows a layered architecture with clear separation of concerns:


PRESENTATION LAYER
BookController
- Create Book
- Get Books (filtered)
- Get Book by ID
- Update Book
- Delete Book
BorrowingController
- Borrow Book
- Return Book
- Get Borrowing History by Book
- Get Borrowing History by Borrower
APPLICATION LAYER
BookService
- Business Logic
- Validation
- Search Logic
BorrowingService
- Borrow/Return Logic
- Status Management
- History Retrieval


Validation Layer
- Bean Validation (DTOs)
- Custom Business Validators
- ISBN Uniqueness Validation
- Borrowing Eligibility Validation
DOMAIN LAYER
LibraryItem (abstract)
- Common fields
- Abstract methods
BorrowingRecord
- Book Reference
- Borrower Name
- Borrow Date
- Return Date
- Status
PhysicalItem
DigitalItem


(abstract)
(abstract)
Book
- author
- isbn
- status
- edition
INFRASTRUCTURE LAYER
Repository Layer
BookRepository
- JPA Repository
- Specifications
BorrowingRecordRepository
- JPA Repository
- Custom Queries



2.2 Technology Stack

Layer	Technology	Version	Purpose
Language	Java	22	Core programming language
Framework	Spring Boot	3.2.4	Application framework
Data Access	Spring Data JPA	3.2.4	Repository abstraction
ORM	Hibernate	6.4.4	Object-relational mapping
Database	PostgreSQL	15.x	Production database
Migrations	Flyway	9.22.3	Schema versioning
Mapping	MapStruct	1.5.5.Final	DTO/Entity conversion
Boilerplate	Lombok	1.18.30	Code generation
Docs	SpringDoc OpenAPI	2.4.0	API documentation
Build	Maven	3.9.x	Build automation

Layer	Technology	Version	Purpose
Unit Testing	JUnit 5	5.10.2	Test framework
Mocking	Mockito	5.10.0	Test doubles
Integration Testing	Testcontainers	1.19.7	Containerized tests
Coverage	JaCoCo	0.8.12	Code coverage
Static Analysis	PMD	3.26.0	Code quality
Formatting	Spotless	2.41.0	Code style enforcement
Containerization	Docker	Latest	Application packaging
CI/CD	GitHub Actions	-	Pipeline automation

2.3 Project Structure

```
library-management-api/
├── src/
│   ├── main/
│   │   ├── java/com/mehdymokhtari/libraryapi/
│   │   │   ├── config/              # Configuration classes
│   │   │   ├── controller/          # REST endpoints
│   │   │   ├── service/             # Business logic
│   │   │   │   ├── impl/            # Service implementations
│   │   │   │   └── validation/      # Business validators
│   │   │   ├── repository/          # Data access
│   │   │   │   └── spec/            # JPA Specifications
│   │   │   ├── model/               # Domain model
│   │   │   │   ├── entity/          # JPA entities
│   │   │   │   ├── dto/             # Data Transfer Objects
│   │   │   │   ├── enums/           # Enumerations
│   │   │   │   └── mapper/          # MapStruct mappers
│   │   │   ├── exception/           # Global error handling
│   │   │   ├── filter/              # Search filters
│   │   │   └── util/                # Utilities
│   │   └── resources/
│   │       ├── application.yml      # Main configuration
│   │       ├── application-docker.yml # Docker config
│   │       └── db/migration/        # Flyway migrations
│   └── test/                        # Test sources
├── .github/workflows/               # CI/CD pipeline
├── Dockerfile                       # Docker image definition
```

```
├── docker-compose.yml                # Service orchestration
├── pom.xml                           # Maven dependencies
└── README.md                         # Project documentation
```

---

3. Domain Model

3.1 Inheritance Hierarchy

The system uses a joined inheritance strategy to support multiple library item types while maintaining a clean domain model:

```
LibraryItem (abstract)
├── id: Long
├── title: String
├── publicationYear: Integer
├── deleted: boolean
├── createdAt: LocalDateTime
├── updatedAt: LocalDateTime
├── version: Long (Optimistic Locking)
├── + borrow(): void (abstract)
├── + returnItem(): void (abstract)
├── + isAvailable(): boolean (abstract)
└── + isBorrowed(): boolean (abstract)

    ├── PhysicalItem (abstract)
    │   └── + status: PhysicalItemStatus
    │
    │       └── Book (concrete) ✅ Currently Implemented
    │           ├── author: String
    │           ├── isbn: String (UNIQUE)
    │           ├── edition: Integer
    │           └── publisher: String (optional)
    │
    │       └── Magazine (abstract) ⏸️ Future
    │           ├── issueNumber: String
    │           ├── editor: String
    │           └── issn: String
    │
    │       └── Dvd (abstract) ⏸️ Future
    │           ├── director: String
```

```
│           ├── duration: Integer
│           └── genre: String
│
└── DigitalItem (abstract)
    └── + format: String
    └── + fileSize: Long

        └── Ebook (abstract) ⏸️ Future
            ├── author: String
            ├── isbn: String
            ├── edition: Integer
            └── downloadLink: String

        └── AudioBook (abstract) ⏸️ Future
            ├── author: String
            ├── narrator: String
            ├── duration: Integer
            └── isbn: String
```

3.2 Entity Details

3.2.1 Book Entity

File: `src/main/java/com/mehdymokhtari/libraryapi/model/entity/Book.java`

Purpose: Represents a single physical book instance. Extends `PhysicalItem` and implements all abstract methods for book-specific borrowing behavior.

Fields:

- `author` : String (required) - The book's author
- `isbn` : String (required, unique) - International Standard Book Number
- `edition` : Integer (required) - Edition number
- `publisher` : String (optional) - Publisher name
- `status` : BookStatus (required) - AVAILABLE or BORROWED

Key Methods:

- `borrow()` : Changes status to BORROWED
- `returnItem()` : Changes status to AVAILABLE
- `isAvailable()` : Returns true if status is AVAILABLE and not deleted
- `isBorrowed()` : Returns true if status is BORROWED and not deleted

Validation Rules:

- ISBN must be unique across all books
- ISBN format must be valid (10 or 13 digits)
- Edition must be a positive integer

3.2.2 BorrowingRecord Entity

File: `src/main/java/com/mehdymokhtari/libraryapi/model/entity/BorrowingRecord.java`

Purpose: Represents a single borrowing transaction lifecycle from borrow to return.

Fields:

- `id` : Long (auto-generated) - Primary key
- `item` : LibraryItem (required) - Reference to the borrowed item (polymorphic)
- `borrowerName` : String (required, max 100 chars) - Name of the borrower
- `borrowedDate` : LocalDate (required) - Date the item was borrowed
- `returnDate` : LocalDate (optional) - Date the item was returned
- `status` : BorrowingStatus (required) - BORROWED or RETURNED
- `version` : Long - Optimistic locking version

Key Methods:

- `markReturned()` : Sets return date to today and status to RETURNED
- `isActive()` : Returns true if status is BORROWED

Validation Rules:

- Borrower name is required and cannot exceed 100 characters
- Borrowed date is automatically set to current date
- Only one active borrowing per item (enforced by unique index)

3.3 Database Schema

Migration Files:

- `V1__create_book_table.sql`
- `V2__create_borrowing_record_table.sql`
- `V3__add_unique_active_borrowing_constraints.sql`

Location: `src/main/resources/db/migration/`

3.3.1 Library Items Table

Purpose: Root table for all library items (books, magazines, DVDs, etc.)

```
Table: library_items
┌──────────────────┬──────────────────────────┬──────────────────────────────
──┐
│ Column           │ Type                     │ Constraints
  │
├──────────────────┼──────────────────────────┼──────────────────────────────
──┤
│ id               │ BIGSERIAL                │ PRIMARY KEY
  │
│ title            │ VARCHAR(255)             │ NOT NULL
  │
│ publication_year │ INT                      │ NOT NULL
  │
│ deleted          │ BOOLEAN                  │ DEFAULT FALSE
  │
│ item_type        │ VARCHAR(20)              │ NOT NULL
  │
│ created_at       │ TIMESTAMP                │ DEFAULT CURRENT_TIMESTAMP
  │
│ updated_at       │ TIMESTAMP                │ DEFAULT CURRENT_TIMESTAMP
  │
│ version          │ BIGINT                   │ DEFAULT 0
  │
└──────────────────┴──────────────────────────┴──────────────────────────────
──┘

Indexes:
- idx_library_items_title ON library_items(title)
- idx_library_items_not_deleted ON library_items(id) WHERE deleted = false
```

3.3.2 Physical Items Table

Purpose: Intermediate table for physical items (books, magazines, DVDs)

```
Table: physical_items
┌──────────────────┬──────────────┬──────────────────────────────┐
│ Column           │ Type         │ Constraints                  │
├──────────────────┼──────────────┼──────────────────────────────┤
│ item_id          │ BIGINT       │ PRIMARY KEY                  │
```

```
└──────────────────┴──────────────┴──────────────────────────────────┘
```

```
Foreign Key:
- fk_physical_items_root REFERENCES library_items(id) ON DELETE CASCADE
```

3.3.3 Books Table

Purpose: Concrete table for Book entities with book-specific fields

```
Table: books
```

Column	Type	Constraints
id	BIGINT	PRIMARY KEY
status	VARCHAR(20)	NOT NULL
author	VARCHAR(255)	NOT NULL
isbn	VARCHAR(17)	NOT NULL, UNIQUE
edition	INT	NOT NULL
publisher	VARCHAR(255)	

```
Foreign Key:
- fk_books_physical REFERENCES physical_items(item_id) ON DELETE CASCADE

Indexes:
- idx_books_isbn ON books(isbn) UNIQUE
- idx_books_author ON books(author)
- idx_books_status ON books(status)
```

3.3.4 Borrowing Records Table

Purpose: Tracks all borrowing transactions

Table: borrowing_records

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
item_id	BIGINT	NOT NULL
borrower_name	VARCHAR(100)	NOT NULL
borrowed_date	DATE	NOT NULL
return_date	DATE	
status	VARCHAR(20)	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
version	BIGINT	DEFAULT 0

Foreign Key:
- fk_borrowing_record_item REFERENCES library_items(id) ON DELETE CASCADE

Unique Index:
- uk_borrowing_active_item ON borrowing_records(item_id) WHERE status = 'BORROWED'

Indexes:
- idx_borrowing_item_status ON borrowing_records(item_id, status)
- idx_borrowing_borrower ON borrowing_records(borrower_name)

---

4. API Documentation

4.1 Book Management Endpoints

Controller File:
`src/main/java/com/mehdymokhtari/libraryapi/controller/BookController.java`

4.1.1 Create Book

Endpoint: `POST /api/v1/books`

Request DTO: `BookRequest`
(`src/main/java/com/mehdymokhtari/libraryapi/model/dto/request/BookRequest.java`)

Request Body:

```
{
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "isbn": "9780132350884",
    "publicationYear": 2008
}
```

Validation Rules:

- `title` : Required, max 255 characters
- `author` : Required, max 255 characters
- `isbn` : Required, valid ISBN format (10 or 13 digits)
- `publicationYear` : Required, between 1450 and 2100

Responses:

- `201 Created` : Book successfully created
- `400 Bad Request` : Validation failed
- `409 Conflict` : ISBN already exists

4.1.2 Get All Books

Endpoint: `GET /api/v1/books`

Query Parameters:

Parameter	Type	Description

Parameter	Type	Description
`title`	String	Filter by title (partial match)
`author`	String	Filter by author (partial match)
`year`	Integer	Filter by publication year
`status`	String	Filter by status (AVAILABLE/BORROWED)
`page`	Integer	Page number (0-based)
`size`	Integer	Items per page
`sort`	String	Sort by field (e.g., `id,asc`)

Filter DTO: `BookFilter`
(`src/main/java/com/mehdymokhtari/libraryapi/filter/BookFilter.java`)

Response DTO: `PagedResponse<T>`
(`src/main/java/com/mehdymokhtari/libraryapi/model/dto/response/PagedResponse.java`)

Response:

```
{
    "success": true,
    "message": "Books retrieved successfully",
    "data": {
        "content": [...],
        "pageNumber": 0,
        "pageSize": 20,
        "totalElements": 15,
        "totalPages": 1,
        "last": true
    }
}
```

4.1.3 Get Book by ID

Endpoint: `GET /api/v1/books/{id}`

Response DTO: `BookResponse`
(`src/main/java/com/mehdymokhtari/libraryapi/model/dto/response/BookResponse.java`)

Responses:

- `200 OK` : Book found
- `404 Not Found` : Book not found

4.1.4 Update Book

Endpoint: `PUT /api/v1/books/{id}`

Request DTO: `BookUpdateRequest` (`src/main/java/com/mehdymokhtari/libraryapi/model/dto/request/BookUpdateRequest.java`)

Request Body:

```
{
    "title": "Clean Code Updated",
    "author": "Robert C. Martin",
    "publicationYear": 2009
}
```

Validation Rules:

- `title` : Required, max 255 characters
- `author` : Required, max 255 characters
- `publicationYear` : Required, between 1450 and 2100

Note: ISBN cannot be updated (business rule enforced in `BookMapper` where `isbn` is ignored).

Responses:

- `200 OK` : Book updated successfully
- `400 Bad Request` : Validation failed
- `404 Not Found` : Book not found

4.1.5 Delete Book

Endpoint: `DELETE /api/v1/books/{id}`

Rules:

- Book must not be borrowed (checked in `BookValidationService.validateBookNotBorrowed()`)

- Soft delete only (sets `deleted = true` in `Book.delete()` method)

Responses:

- `200 OK` : Book deleted successfully
- `404 Not Found` : Book not found
- `409 Conflict` : Book is currently borrowed

4.2 Borrowing Management Endpoints

Controller File:
`src/main/java/com/mehdymokhtari/libraryapi/controller/BorrowingController.java`

4.2.1 Borrow a Book

Endpoint: `POST /api/v1/borrowings/borrow`

Request DTO: `BorrowRequest` (`src/main/java/com/mehdymokhtari/libraryapi/model/dto/request/BorrowRequest.java`)

Request Body:

```
{
    "itemId": 1,
    "borrowerName": "John Doe"
}
```

Validation Rules:

- `itemId` : Required, positive number
- `borrowerName` : Required, max 100 characters

Business Rules (enforced in `BorrowingServiceImpl`):

- Book must be available (`item.isAvailable()`)
- Book status changes to `BORROWED` (`item.borrow()`)
- Borrowing record is created with `borrowedDate = LocalDate.now()`

Responses:

- `200 OK` : Book borrowed successfully

- `400 Bad Request` : Validation failed
- `404 Not Found` : Book not found
- `409 Conflict` : Book is not available

4.2.2 Return a Book

Endpoint: `POST /api/v1/borrowings/return`

Request DTO: `ReturnRequest` (`src/main/java/com/mehdymokhtari/libraryapi/model/dto/request/ReturnRequest.java`)

Request Body:

```
{
    "itemId": 1
}
```

Business Rules (enforced in `BorrowingServiceImpl`):

- Book must be currently borrowed (`item.isBorrowed()`)
- Active borrowing record must exist
- Book status changes to `AVAILABLE` (`item.returnItem()`)
- Borrowing record is updated with `returnDate = LocalDate.now()` and `status = RETURNED`

Responses:

- `200 OK` : Book returned successfully
- `400 Bad Request` : Validation failed
- `404 Not Found` : Book not found
- `409 Conflict` : Book is not currently borrowed

4.2.3 Get Borrowing History by Book

Endpoint: `GET /api/v1/borrowings/book/{bookId}`

Response DTO: `BorrowingRecordResponse` (`src/main/java/com/mehdymokhtari/libraryapi/model/dto/response/BorrowingRecordResponse.java`)

Responses:

- `200 OK` : History retrieved successfully
- `404 Not Found` : Book not found

4.2.4 Get Borrowing History by Borrower

Endpoint: `GET /api/v1/borrowings/borrower/{name}`

Responses:

- `200 OK` : History retrieved successfully

---

5. Business Logic

5.1 Service Layer

5.1.1 Book Service

File:
`src/main/java/com/mehdymokhtari/libraryapi/service/impl/BookServiceImpl.java`

Key Methods and Business Logic:

Method	Business Logic	Validation
`createBook()`	ISBN must be unique; status set to AVAILABLE	`BookValidationService.validateCreateBook()`
`getAllBooks()`	Apply filters and pagination	None
`getBookById()`	Book must exist	`BookValidationService.validateAndGetBook()`
`updateBook()`	Book must exist; ISBN cannot be updated	`BookValidationService.validateUpdateBook()`
`deleteBook()`	Book must exist and not be borrowed; soft delete	`BookValidationService.validateDeleteBook()` and `validateBookNotBorrowed()`

5.1.2 Borrowing Service

File:
`src/main/java/com/mehdymokhtari/libraryapi/service/impl/BorrowingServiceI`

mpl.java

Key Methods and Business Logic:

Method	Business Logic	Validation
`borrowItem()`	Book must be available; create borrowing record; update book status	`BorrowingValidator.validateItemAvailable()`
`returnItem()`	Book must be borrowed; active record must exist; update record and status	`BorrowingValidator.validateItemBorrowed()`
`getBorrowingHistoryByItem()`	Book must exist	`LibraryItemRepository.existsByIdAndDeletedFalse()`
`getBorrowingHistoryByBorrower()`	Return records matching borrower name	None

5.2 Validation Layer

5.2.1 Bean Validation (DTO Level)

Files: All request DTOs in `model/dto/request/`

DTO	Validation Annotations	Purpose
`BookRequest`	`@NotBlank`, `@Size`, `@Pattern`, `@NotNull`, `@Min`, `@Max`	Validate book creation data
`BookUpdateRequest`	`@NotBlank`, `@Size`, `@NotNull`, `@Min`, `@Max`	Validate book update data
`BorrowRequest`	`@NotNull`, `@Positive`, `@NotBlank`, `@Size`	Validate borrow request
`ReturnRequest`	`@NotNull`, `@Positive`	Validate return request

5.2.2 Business Validation (Service Level)

Files: `service/validation/`

Validator	Methods	Purpose
`BookValidationService`	`validateCreateBook()`, `validateUpdateBook()`, `validateDeleteBook()`, `validateAndGetBook()`, `validateBookNotBorrowed()`, `validateIsbnUniqueForUpdate()`	Book-specific business rules

Validator	Methods	Purpose
`BorrowingValidator`	`validateBorrowerName()`, `validateItemAvailable()`, `validateItemBorrowed()`, `validateAndGetActiveBorrowing()`, `validateItemNotAlreadyBorrowed()`	Borrowing-specific business rules
`BookStatusValidator`	`validateStatus()`, `validateBookCanBeBorrowed()`, `validateBookCanBeReturned()`	Status transition validation

5.3 Search Logic

File: `src/main/java/com/mehdymokhtari/libraryapi/repository/spec/BookSpecification.java`

Search uses Spring Data JPA Specifications to build dynamic WHERE clauses:

Method	Purpose
`withFilters(BookFilter)`	Combines all filters into a single specification
`hasTitle(String)`	Partial match on title (case-insensitive)
`hasAuthor(String)`	Partial match on author (case-insensitive)
`hasPublicationYear(Integer)`	Exact match on publication year
`hasStatus(BookStatus)`	Exact match on status
`isNotDeleted()`	Filters out soft-deleted books

---

6. Exception Handling

6.1 Exception Hierarchy

Files: `src/main/java/com/mehdymokhtari/libraryapi/exception/`

```
Exception (java.lang)
    └── RuntimeException
        ├── ResourceNotFoundException (404)    - Resource not found
        ├── BookNotAvailableException (409)    - Book unavailable/conflict
        ├── InvalidOperationException (400)    - Invalid operation
        ├── BusinessException (422)            - Business rule violation
```

└── ItemNotBorrowedException (409) - Item not borrowed for return

6.2 Global Exception Handler

File: `src/main/java/com/mehdymokhtari/libraryapi/exception/GlobalExceptionHandler.java`

The `GlobalExceptionHandler` provides centralized error handling:

Exception	HTTP Status	When Thrown
`ResourceNotFoundException`	404	Book/item not found
`BookNotAvailableException`	409	Book already borrowed or not available
`ItemNotBorrowedException`	409	Trying to return an item that isn't borrowed
`InvalidOperationException`	400	Invalid operation
`BusinessException`	422	General business rule violation
`MethodArgumentNotValidException`	400	Bean Validation failure
`ConstraintViolationException`	400	Validation constraint violation
`DataIntegrityViolationException`	409	Database constraint violation
`Exception` (default)	500	Unhandled exception

6.3 Error Response Format

File: `src/main/java/com/mehdymokhtari/libraryapi/exception/ErrorResponse.java`

```
{
    "status": 409,
    "error": "Conflict",
    "message": "Item with ID 1 is not available for borrowing",
    "path": "/api/v1/borrowings/borrow",
    "timestamp": "2026-07-07T10:30:00.123456789",
    "validationErrors": {
        "field1": "Validation error message"
```

```
    }
}
```

---

7. Testing Strategy

7.1 Test Types

Test Type	Files	Framework	Purpose
Unit Tests	`controller/*Test.java`, `service/*Test.java`	JUnit 5, Mockito	Test individual components in isolation
Integration Tests	`repository/*Test.java`	Testcontainers	Test with real PostgreSQL container
End-to-End Tests	`integration/LibraryManagementIntegrationTest.java`	Spring Boot Test	Test complete API flows
Coverage	-	JaCoCo	Measure test coverage

7.2 Test Coverage Goals

Metric	Target
Instruction Coverage	≥ 80%
Branch Coverage	≥ 70%
Line Coverage	≥ 80%

Configuration: `jacoco-maven-plugin` in `pom.xml` enforces these thresholds.

7.3 Test Structure

```
src/test/java/com/mehdymokhtari/libraryapi/
├── controller/
│   ├── BookControllerTest.java          # Book API endpoint tests
│   ├── BorrowingControllerTest.java     # Borrowing API endpoint tests
│   └── BaseControllerTest.java          # Base controller test
configuration
├── service/
```

```
│   ├── BookServiceTest.java              # Book service business logic
tests
│   └── BorrowingServiceTest.java         # Borrowing service business
logic tests
├── repository/
│   ├── BookRepositoryTest.java           # Book repository database tests
│   └── BorrowingRecordRepositoryTest.java # Borrowing record database
tests
├── integration/
│   └── LibraryManagementIntegrationTest.java # End-to-end API tests
└── resources/
    └── application-test.yml              # Test configuration
```

---

8. CI/CD Pipeline

8.1 GitHub Actions Workflow

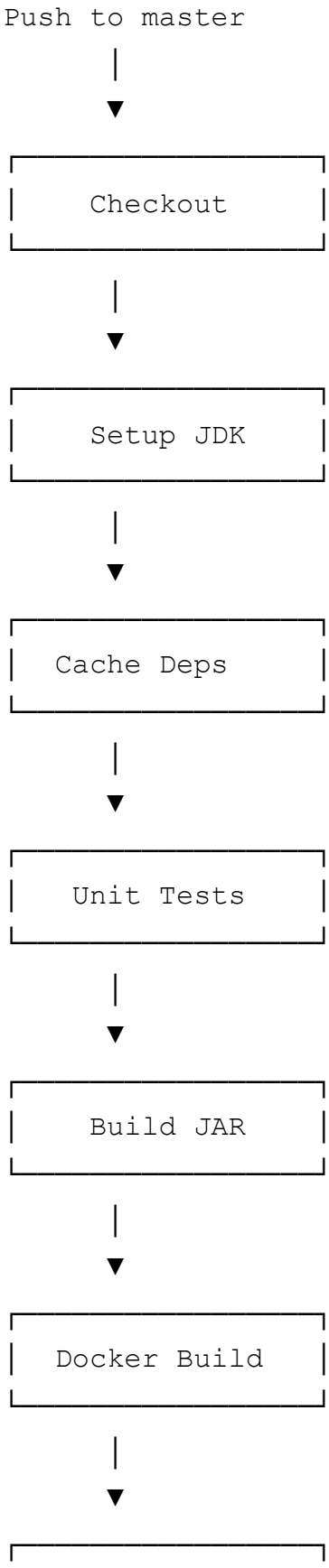
File: `.github/workflows/ci.yml`

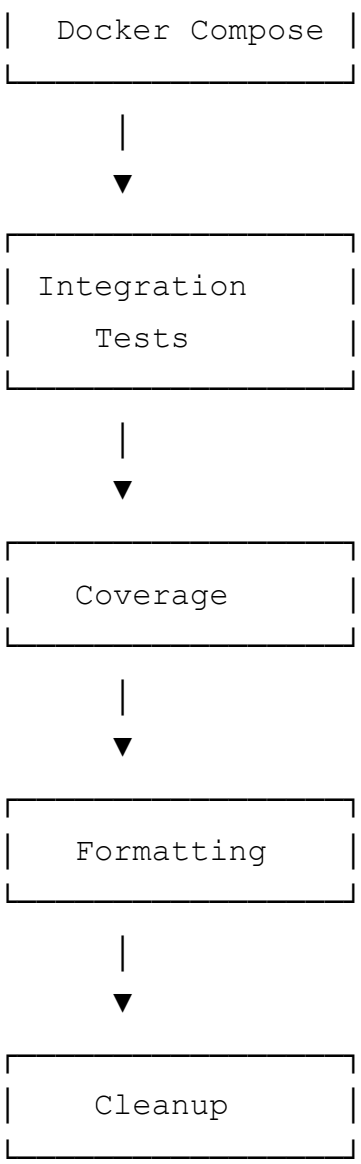
The CI/CD pipeline runs on every push to the `master` branch:

Stage	Command	Purpose
Checkout	`actions/checkout@v4`	Fetch source code
Setup	`actions/setup-java@v4`	Install JDK 22 with caching
Dependency Cache	`actions/cache@v4`	Cache Maven dependencies
Unit Tests	`mvn test`	Run all unit tests
Build	`mvn clean package -DskipTests`	Package application JAR
Docker Build	`docker build`	Build optimized container image
Start Services	`docker compose up -d`	Start PostgreSQL and app containers
Integration Tests	`docker compose exec -T app mvn verify`	Run tests against container
Coverage	`docker compose exec -T app mvn jacoco:report`	Generate JaCoCo report
Upload Artifact	`actions/upload-artifact@v4`	Upload coverage report

Stage	Command	Purpose
Formatting	`mvn spotless:check`	Enforce Google Java Format
Cleanup	`docker compose down`	Stop all containers

8.2 Pipeline Flow





---

9. Deployment

9.1 Docker Setup

9.1.1 Dockerfile

File: `Dockerfile`

Multi-stage build for optimized image:

```
Stage 1: Build
  FROM maven:3.9.9-eclipse-temurin-22-alpine
  - Copy pom.xml and download dependencies
  - Copy source code and build JAR

Stage 2: Runtime
  FROM eclipse-temurin:22-jre-alpine
```

```
- Install curl for health checks
- Create non-root user for security
- Copy JAR from build stage
- Set JVM options for container optimization
- Expose port 8080
- Set entrypoint with JVM options
```

Key Features:

- Non-root user (`spring`) for security
- Multi-stage build for minimal image size
- JVM optimized for containers
- Health check ready with curl

9.1.2 Docker Compose

File: `docker-compose.yml`

Orchestrates three services:

Service	Image	Purpose
`postgres`	`postgres:15-alpine`	PostgreSQL database with health checks
`app`	`library-api:${IMAGE_VERSION:-1.0.0}`	Spring Boot application
`pgadmin`	`pgadmin4:latest`	Database GUI (dev profile only)

Network: `library-network` (bridge)

Volumes: `postgres_data` (persistent database storage)

9.2 Start Script

File: `scripts/start.sh`

Command	Action
`./scripts/start.sh run`	Apply formatting, run tests, build Docker image, start stack
`./scripts/start.sh clean`	Stop containers and remove volumes

Features:

- Automatically extracts version from `pom.xml`

- Runs `spotless:apply`, `verify`, `pmd:check`
- Builds Docker image with dynamic version tagging
- Starts full stack with Docker Compose

9.3 Cloud Deployment Options

Provider	Service	Best For
AWS	ECS/Fargate	Serverless container deployment with auto-scaling
Azure	Container Apps	Managed container orchestration on Azure
GCP	Cloud Run	Fully managed serverless platform
Heroku	Container Registry	Simple deployment for demos and prototyping

---

10. Performance Optimizations

Optimization	Implementation	File Location	Benefit
Connection Pooling	HikariCP (default in Spring Boot)	`application.yml`	Reduced database connection overhead
Lazy Loading	`fetch = FetchType.LAZY`	`BorrowingRecord.java`	Optimized entity loading
Query Optimization	JPA Specifications	`BookSpecification.java`	Efficient dynamic queries
Pagination	Spring Data Pageable	`BookController.java`	Reduced data transfer
Batch Processing	`batch_size = 20`	`application.yml`	Efficient bulk operations
Indexing	Database indexes	`V1__create_book_table.sql`, `V2__create_borrowing_record_table.sql`	Fast search queries
Soft Delete	`deleted` flag	`LibraryItem.java`	Preserves data without performance impact
Optimistic Locking	`@Version`	`LibraryItem.java`, `BorrowingRecord.java`	Prevents concurrent update conflicts

---

11. Security Considerations

Aspect	Implementation	File Location
Input Validation	Bean Validation on all DTOs	`model/dto/request/*.java`
SQL Injection	JPA/Hibernate parameterized queries	All repository methods
Transaction Management	`@Transactional`	`service/impl/*ServiceImpl.java`
Exception Handling	Global exception handler	`GlobalExceptionHandler.java`
Environment Configs	Separate configs for dev, test, docker, prod	`application*.yml`
CORS	Configured for API access	`AppConfig.java`
Monitoring	Actuator endpoints	`application.yml`
Data Retention	Soft delete for audit trail	`LibraryItem.delete()`

---

12. Future Enhancements

Enhancement	Description	Priority
Authentication	JWT-based user authentication with roles	High
Analytics Dashboard	Borrowing statistics and popular books	High
Email Notifications	Overdue reminders and confirmations	Medium
Redis Caching	Faster search results and reduced DB load	Medium
Reservation System	Allow users to reserve unavailable books	Medium
Complete Inheritance	Add Magazine, Ebook, AudioBook support	Low
GraphQL	Alternative flexible query interface	Low
WebSocket	Real-time availability updates	Low
Bulk Operations	CSV/Excel import/export	Low
Elasticsearch	Advanced full-text search capabilities	Low

---

13. Appendices

Appendix A: Environment Variables

Variable	Description	Default
`SPRING_PROFILES_ACTIVE`	Active profile	`docker`
`SPRING_DATASOURCE_URL`	Database connection URL	`jdbc:postgresql://postgres:5432/library_db`
`SPRING_DATASOURCE_USERNAME`	Database username	`library_user`
`SPRING_DATASOURCE_PASSWORD`	Database password	`library_password`
`SPRING_FLYWAY_ENABLED`	Flyway migration enabled	`true`
`IMAGE_VERSION`	Docker image version	`1.0.0`

Appendix B: Useful Commands

Command	Purpose
`./scripts/start.sh run`	Run full stack with automation script
`./scripts/start.sh clean`	Reset environment
`docker compose up -d`	Start services manually
`docker compose logs -f app`	View application logs
`mvn clean verify`	Build and test
`mvn spotless:apply`	Fix formatting
`mvn jacoco:report`	Generate coverage report
`mvn pmd:check`	Run static analysis

Appendix C: File References

File	Purpose	Location
`Book.java`	Book entity	`model/entity/`
`BorrowingRecord.java`	Borrowing record entity	`model/entity/`
`BookController.java`	Book REST API	`controller/`

File	Purpose	Location
`BorrowingController.java`	Borrowing REST API	`controller/`
`BookServiceImpl.java`	Book business logic	`service/impl/`
`BorrowingServiceImpl.java`	Borrowing business logic	`service/impl/`
`BookSpecification.java`	Dynamic search queries	`repository/spec/`
`GlobalExceptionHandler.java`	Exception handling	`exception/`
`Dockerfile`	Docker image definition	Root
`docker-compose.yml`	Service orchestration	Root
`ci.yml`	CI/CD pipeline	`.github/workflows/`